Diagnosing long running transactions

This runbook documents the methodology for diagnosing `PatroniLongRunningTransactionDetected` alerts.

Background

Long-running transactions are one of the main drivers for `pg_txid_xmin_age`, and can result in severe performance degradation if left unaddressed.

Getting the current query

The first step is to figure out what the long-running transaction is doing. The alert will tell you which host to look at. From there, it is possible to get a `query`, `pid`, `client_addr`, and `client_port`:

```
iwiedler@patroni-main-2004-01-db-gprd.c.gitlab-production.internal:~$ sudo gitlab-psql

gitlabhq_production=# \x
Expanded display is on.

gitlabhq_production=# \pset pager 0
Pager usage is off.

gitlabhq_production=# select *,
  now(),
  now() - query_start as query_age,
  now() - xact_start as xact_age
from pg_stat_activity
where
  state != 'idle'
  and backend_type != 'autovacuum worker'
  and xact_start < now() - '60 seconds'::interval
order by now() - xact_start desc nulls last
;
```

Getting the pgbouncer instance

Usually postgres connections go through a pgbouncer. The `client_addr` and `client_port` will tell you which one.

In this case it was `pgbouncer-01-db-gprd` as discovered via the `client_port` of `42792`:

```
iwiedler@patroni-main-2004-01-db-gprd.c.gitlab-production.internal:~$ sudo netstat -tp | gre

tcp        0    229 patroni-main:postgresql pgbouncer-01-db-g:42792 ESTABLISHED 3889825/post
```

Resolving pgbouncer port to client

Now that we have the pgbouncer address and port, we can log into that pgbouncer box and get the actual client.

This can be done by first running `show sockets` on the pgbouncer admin console, and finding the metadata for the backend port:

```
iwiedler@pgbouncer-01-db-gprd.c.gitlab-production.internal:~$ sudo pgb-console -c 'show sock

 type |     user      |      database       |   state   |     addr      | port  | local_addr
 S    | gitlab        | gitlabhq_production | sv_active | 10.220.21.101 |  5432 | 10.217.4.3
```

Using this information we can then grab the `link` column. And join it against `show clients` to discover the actual client:

```
iwiedler@pgbouncer-01-db-gprd.c.gitlab-production.internal:~$ sudo pgb-console -c 'show clie

 type |   user    |      database       | state  |     addr      | port  | local_addr | loca
 C    | gitlab    | gitlabhq_production | active | 10.218.5.2    | 49266 | 10.217.4.5 |
```

Double checking connections, and this tells us that it's `console-01-sv-gprd`:

```
iwiedler@pgbouncer-01-db-gprd.c.gitlab-production.internal:~$ sudo netstat -tp | grep 49266

tcp        0     35 10.217.4.5:6432         console-01-sv-gpr:49266 ESTABLISHED 19532/pgbour
```

Resolving client address and port to process and user

Now that we've confirmed it's a console user, we can look up which process on the console host holds that connection:

```
iwiedler@console-01-sv-gprd.c.gitlab-production.internal:~$ sudo ss -tp | grep 49266

ESTAB      0      194             10.218.5.2:49266              10.217.4.5:6432  users:(("ru
```

And we can feed the pid into `pstree` to see the process hierarchy:

```
systemd(1)  sshd(1369)  sshd(3225836)  sshd(3226021,arihant-rails)  bash(3226022)  script(3226
```

In this case it was user `arihant-rails` who was running a script via rails console.

Cancelling the query

In most cases, we will want to cancel the query. If it's a single long-running query, this can be done via `pg_cancel_backend` (passing in the `pid`):

```
gitlabhq_production=# select pg_cancel_backend(<pid>);
 pg_cancel_backend
-------------------
 t
```

However, if we are dealing with a long-running transaction consisting of many short-lived queries, it may be necessary to terminate the backend instead:

```
gitlabhq_production=# select pg_terminate_backend(<pid>);
 pg_terminate_backend
----------------------
 t
```